FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P2

Relatório consolidado

arquitetura e atividades do 2º bimestre

Avaliação individual · 2026.2

Victor Hugo Cano Siveri

Produto: **OpenWave**
https://github.com/fezinhopatta/TraumaTeam

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 13/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura do SaaS 3
3.1 Visão geral e componentes 3
3.2 Onde roda 4
3.3 Fluxo de uma requisição 4
3.4 Fluxo de assinatura 5
3.5 Modelo de dados do domínio 6
3.6 Segurança 7
3.7 Observabilidade, CI/CD e documentação da API 8
3.8 Camadas IaaS, PaaS e SaaS do projeto 8
4 Quadro de entregas 9
5 Atividades realizadas 10
5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe 10
5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve 11
5.3 Atividade 9 — Fork e plano de negócio 13
5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes 14
5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe 17
6 Considerações finais 21
7 Declaração de autoria 21

1 Introdução

A disciplina de Introdução à Computação em Nuvem (ISW055), ministrada na Fatec Pompeia pelo Prof. Allan Lincoln Rodrigues Siriani (@siriani), capacita o estudante na arquitetura, orquestração e segurança de microsserviços em nuvem. No decorrer do semestre, o catálogo didático de filmes de Tom Hanks do 1º bimestre foi transformado e refatorado em um produto de software comercial real: o **OpenWave**.

O **OpenWave** é uma plataforma SaaS de inteligência e monitoramento tático de radiocomunicação voltada para segurança pública e defesa civil. A aplicação resolve a dor da fragmentação de transmissões analógicas de emergência (Bombeiros, SAMU, Defesa Civil), conectando rádio-operadores voluntários (**feeders**) a ouvintes e equipes de apoio (**listeners**). A monetização recorrente via Stripe sustenta o negócio com planos escaláveis: Comunitário Gratuito (com delay e cota de 3 favoritos), **Sentinela Pro** (R$ 9,90/mês, tempo real e favoritos ilimitados) e **Central Tática VIP** (R$ 29,90/mês, downloads de áudio do MinIO e prioridade).

Este relatório constitui a prestação de contas individual da Avaliação P2. Ele documenta na Seção 3 a arquitetura completa do SaaS no ar (microsserviços, rede Docker, fluxos de requisição e assinatura, modelo de dados, segurança RBAC e camadas de nuvem) e consolida nas Seções 4 e 5 as evidências auditáveis das Atividades 7 a 11.

2 Metodologia

As atividades 7 e 8 foram desenvolvidas no repositório inicial (`fezinhopatta/Tom-Hanks-api`). Na atividade 9, com a definição do produto OpenWave, o histórico de versionamento foi espelhado integralmente via `git clone --bare` e `git push --mirror` para o repositório definitivo (`fezinhopatta/TraumaTeam`), preservando a rastreabilidade total dos commits. O sistema roda na stack `open-wave` do Portainer em VM Linux da disciplina, compartilhando a rede bridge interna `app-network` sob o domínio público `https://victor-siveri-isw055.lapps.studio/`.

As evidências foram obtidas a partir do histórico Git no fuso de Brasília (`America/Sao_Paulo` - BRT) e conferidas nas páginas oficiais de commits e issues do GitHub com menção ao professor @siriani. Segredos e chaves de API foram mantidos estritamente nas variáveis da stack, versionando-se apenas o `.env.example`. Para a atividade 11, o código foi versionado na branch `ULTIMA` e validado por uma suíte de 23 testes unitários automatizados cobrindo webhooks, HMAC, idempotência e cotas.

3 Arquitetura do SaaS

A arquitetura do OpenWave foi projetada para garantir desacoplamento, segurança de dados e alta disponibilidade, separando claramente as interfaces públicas expostas na internet dos serviços restritos que operam na rede privada do Docker.

3.1 Visão geral e componentes

A aplicação é composta por microsserviços integrados operando na rede interna `app-network` da stack do Portainer, comunicando-se com serviços gerenciados de persistência externa e com o provedor de pagamentos Stripe.



Figura 1: Topologia de Microsserviços e Componentes do OpenWave

Componente	Responsabilidade	Tecnologia / Imagem	Público?
catalog-service	Telas web (SSR), API REST do negócio, landing page, controle de cotas por plano, documentação Swagger e webhooks Stripe.	Python 3.9 / Flask 3.0 (Dockerfile custom)	Sim
auth-service	Autenticação de usuários, cadastro, redefinição de senha e verificação de papéis RBAC (`/api/check-role`).	Python 3.9 / Flask (porta interna 5000)	Não
log-service	Microsserviço de auditoria e telemetria: ingestão assíncrona (`POST /api/log`) e consulta (`GET /api/logs`).	Python 3.9 / Flask (porta interna 5000)	Não
redis-service	Fila em memória não bloqueante baseada em Redis Streams (`audit_logs`) para buffer de auditoria.	Imagem oficial `redis:alpine` (porta 6379)	Não
minio	Object storage compatível com Amazon S3 para fotos de perfil e áudios gravados de ocorrências de rádio.	Imagem `elestio/minio:latest` (portas 9000/9001)	Não

Componente	Responsabilidade	Tecnologia / Imagem	Público?
MariaDB (Externo)	Banco de dados relacional que persiste usuários, planos, assinaturas, canais, favoritos e eventos.	MariaDB externo (`35.226.64.52:3306`)	Acesso SQL restrito
Stripe API (SaaS)	Processamento de cobrança recorrente, faturas, checkout seguro, portal do cliente e webhooks.	SaaS externo (`api.stripe.com`)	Webhook → App

3.2 Onde roda

A infraestrutura do OpenWave está hospedada em uma máquina virtual Linux mantida pelo professor da disciplina, sob o domínio público unificado `https://victor-siveri-isw055.lapps.studio/`.

O ciclo de vida dos containers é orquestrado pelo Portainer na stack denominada `open-wave`. A stack obtém o código-fonte a partir do repositório oficial `https://github.com/fezinhopatta/TraumaTeam` (branch `ULTIMA`). Os limites de memória não foram explicitados no arquivo `docker-compose.yml` (`mem_limit` ausente), permitindo que os containers compartilhem dinamicamente a memória RAM disponível na máquina virtual conforme a demanda de requisições e ingestão de logs.

A injeção de configurações e segredos é realizada exclusivamente por variáveis de ambiente na definição da stack no Portainer. Nenhuma credencial foi commitada no repositório. As variáveis configuradas são:

- **Banco de dados:** `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`.
- **Sessão e Aplicação:** `SECRET_KEY`, `PORTA_ALUNO`, `APP_URL`.
- **Microsserviços internos:** `AUTH_SERVICE_URL`, `LOG_SERVICE_URL`.
- **Armazenamento S3 MinIO:** `MINIO_ENDPOINT`, `MINIO_PORT`, `MINIO_ROOT_USER`, `MINIO_ROOT_PASSWORD`, `MINIO_BUCKET`, `MINIO_PUBLIC_URL`, `MINIO_API_PORT`, `MINIO_CONSOLE_PORT`.
- **Serviço de E-mail (SMTP):** `MAIL_SERVER`, `MAIL_PORT`, `MAIL_USERNAME`, `MAIL_PASSWORD`, `MAIL_FROM`.
- **Integração Stripe:** `STRIPE_SECRET_KEY`, `STRIPE_PUBLISHABLE_KEY`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_PRICE_ID`.

3.3 Fluxo de uma requisição

Para ilustrar o percurso completo de uma requisição pelo sistema, considera-se a ação fundamental de um usuário: **um ouvinte autenticado tenta adicionar uma frequência de rádio aos seus canais favoritos**.

1. **Navegador do Usuário:** O cliente clica no botão “Favoritar Frequência” na interface web, enviando uma requisição HTTP `POST /api/canais/1/favoritar` contendo o cookie de sessão assinado (`session`).
2. **Roteamento Externo:** A requisição chega à porta 443 do proxy reverso sob o domínio `victor-siveri-isw055.lapps.studio`, que valida o certificado SSL/TLS e repassa o tráfego internamente para a porta 5000 do container `catalog-service`.
3. **Identificação e Sessão:** O Flask descompacta o cookie criptográfico da sessão com a chave `SECRET_KEY` e recupera o identificador do usuário (`user_id`).
4. **Validação de Papel (RBAC):** O `catalog-service` realiza uma chamada HTTP interna para o microsserviço de autenticação (`GET http://auth-service:5000/api/check-role/<user_id>`). O `auth-service` confirma que o usuário possui papel válido (`listener` ou `admin`).

5. **Avaliação de Direitos e Cotas SaaS:** O app invoca a função `obter_dados_assinatura(user_id)`, executando uma consulta `SELECT plano, status_assinatura FROM usuarios` no MariaDB externo.
6. **Controle de Limite:** Se o usuário pertencer ao plano comunitário (`free`), o sistema consulta a contagem de frequências já adicionadas na tabela `canais_favoritos`. Caso o total seja igual ou superior a 3:
   - A requisição é interrompida.
   - O sistema despacha evento de bloqueio para o `log-service`.
   - É renderizada e retornada a página educativa de limite atingido com status HTTP 403 Forbidden (`templates/limite.html`), exibindo convite para upgrade para o plano Sentinela Pro.
7. **Persistência no Banco:** Caso o limite não tenha sido violado (ou o usuário seja assinante ativo `pro` ou `central`), a instrução SQL `INSERT INTO canais_favoritos (usuario_id, canal_id) VALUES (...)` é executada no MariaDB.
8. **Telemetria e Auditoria:** O `catalog-service` despacha uma requisição assíncrona HTTP `POST http://log-service:5000/api/log` informando a ação `FAVORITAR_CANAL` e os metadados da transação.
9. **Ingestão no Redis:** O `log-service` grava o registro no stream `audit_logs` do container Redis via comando `XADD`.
10. **Resposta ao Cliente:** O `catalog-service` retorna ao navegador a resposta HTTP 200 com JSON confirmando o sucesso da operação e atualizando o estado do componente visual.

3.4 Fluxo de assinatura

O ciclo de vida da assinatura no OpenWave foi arquitetado para operar de maneira desacoplada e 100% autônoma pelo cliente, sem intervenção manual de administradores.



Figura 2: Ciclo de Vida da Assinatura: Checkout, Webhooks, Idempotência e Autonomia

O fluxo obedece às seguintes etapas:

1. **Criação da Sessão de Checkout:** O usuário clica em "Assinar Sentinela Pro" (`GET /assinatura/checkout?plano=pro`). O backend cria uma sessão no Stripe no modo de assinatura recorrente (`mode='subscription'`), definindo `metadata={'user_id': user_id, 'plano': 'pro'}` e URLs de retorno (`/perfil`), redirecionando o navegador para o Stripe Checkout seguro.
2. **Pagamento no Stripe:** O cliente insere seus dados e conclui a assinatura com cartão de teste (`4242 4242 4242 4242`).

3. **Disparo do Webhook:** O Stripe envia uma requisição `POST /webhook/stripe` para o domínio público do OpenWave com cabeçalho `Stripe-Signature`.
4. **Validação Criptográfica:** O `catalog-service` intercepta o payload bruto e invoca `stripe.Webhook.construct_event(payload, sig, STRIPE_WEBHOOK_SECRET)`. Se a assinatura HMAC SHA-256 for inválida ou o segredo não corresponder, a requisição é rejeitada com HTTP 400 Bad Request, impedindo ataques de forjamento.
5. **Controle Estrito de Idempotência:** O backend consulta a tabela relacional `stripe_eventos` pelo `event_id`. Se o identificador já existir no banco, a mensagem é classificada como duplicata (retransmissão padrão da rede do Stripe), o sistema retorna imediatamente HTTP 200 e aborta a execução, prevenindo alterações redundantes. Caso seja a primeira recepção, o `event_id` é registrado na tabela.
6. **Atualização de Plano e Entitlements:** Ao validar o evento `checkout.session.completed` ou `customer.subscription.updated`, o sistema atualiza a tabela `usuarios`: define `plano = 'pro'`, `status_assinatura = 'active'`, `stripe_customer_id` e `stripe_subscription_id`. A partir deste milissegundo, qualquer requisição do usuário desbloqueia streaming sem delay e favoritos ilimitados.
7. **Portal do Cliente e Autonomia:** Em sua página de perfil (`/perfil`), o assinante tem acesso direto ao Stripe Customer Portal (`/assinatura/portal`), onde pode gerenciar formas de pagamento, baixar recibos fiscais e trocar de plano. O cancelamento autônomo (`/assinatura/cancelar`) agenda o término da cobrança no final do período no Stripe (`cancel_at_period_end`).
8. **Tratamento de Inadimplência e Downgrade:** Caso ocorra falha de pagamento (`invoice.payment_failed`) ou cancelamento consumado (`customer.subscription.deleted`), o webhook atualiza o status do usuário para `past_due` ou `canceled`. A função `obter_dados_assinatura()` detecta a inadimplência e rebaixa imediatamente os direitos do usuário para o plano comunitário gratuito, aplicando as travas de favoritos e delay.

3.5 Modelo de dados do domínio

O modelo relacional foi estruturado no MariaDB em 6 tabelas interdependentes com chaves estrangeiras configuradas com integridade referencial e deleção em cascata (`ON DELETE CASCADE`).

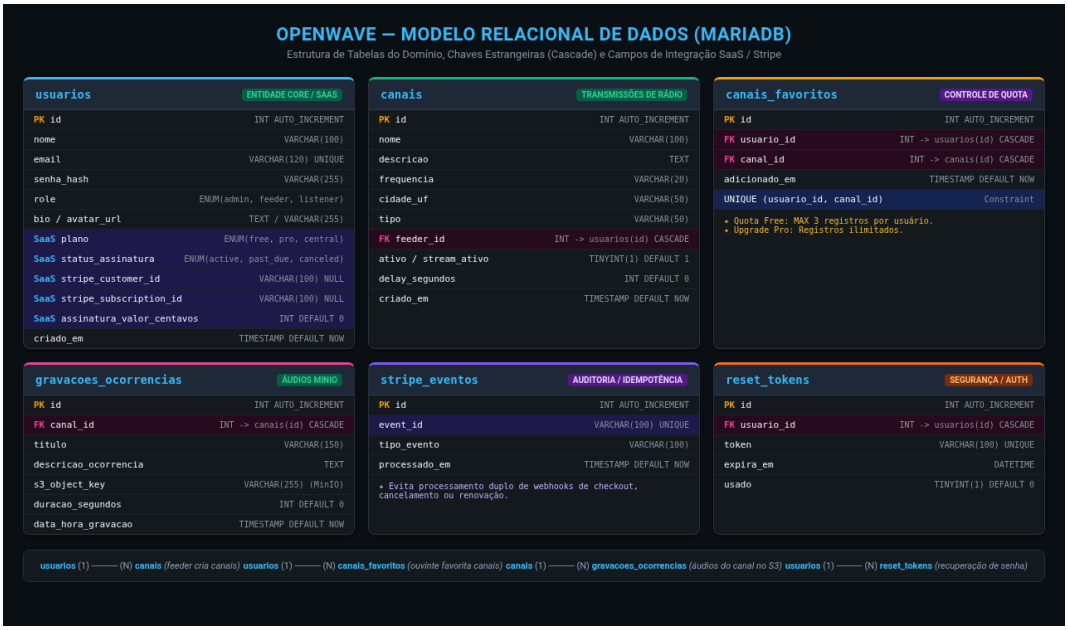


Figura 3: Diagrama Entidade-Relacionamento do OpenWave (MariaDB)

As tabelas e seus papéis no negócio são:

- **`usuarios`:** Entidade central do sistema. Contém credenciais (`id`, `nome`, `email`, `senha_hash`), papel no RBAC (`role`: `admin`, `feeder`, `listener`), perfil (`bio`, `avatar_url`), dados do SaaS (`plano`: `free`, `pro`, `central`), estado de pagamento (`status_assinatura`: `active`, `past_due`, `canceled`), identificadores de integração (`stripe_customer_id`, `stripe_subscription_id`) e valor da assinatura (`assinatura_valor_centavos`).
- **`canais`:** Frequências de rádio cadastradas por voluntários. Armazena `id`, `nome`, `descricao`, `frequencia`, `cidade_uf`, `tipo`, chave estrangeira `feeder_id` apontando para `usuarios(id)`, flags de atividade (`ativo`, `stream_ativo`) e `delay_segundos`.
- **`canais_favoritos`:** Tabela associativa que viabiliza o controle de cotas do SaaS. Armazena `id`, `usuario_id`, `canal_id`, data de criação e uma restrição de unicidade composta `UNIQUE (usuario_id, canal_id)`.
- **`gravacoes_ocorrencias`:** Acervo de áudio dos incidentes capturados. Possui `id`, `canal_id` (FK para `canais`), `titulo`, `descricao_ocorrencia`, a chave de objeto no MinIO (`s3_object_key`) e a duração em segundos.
- **`stripe_eventos`:** Tabela técnica responsável pela garantia de idempotência no recebimento de webhooks do Stripe, registrando `id`, `event_id` (UNIQUE), `tipo_evento` e timestamp.
- **`reset_tokens`:** Tabela de segurança para recuperação de senhas, contendo `usuario_id`, `token` criptográfico, `expira_em` e flag booleana `usado`.

3.6 Segurança

O modelo de segurança do OpenWave baseia-se no princípio do menor privilégio, separação estrita de papéis via RBAC e isolamento multi-tenant a nível de banco de dados.

Ação no Sistema	admin	feeder	listener
Ouvir canais públicos e reproduzir streaming	Sim	Sim	Sim
Favoritar canais (dentro da cota do seu plano)	Sim	Sim	Sim
Cadastrar novo canal e transmitir frequência	Sim	Sim	Não
Fazer upload de gravações de áudio no MinIO	Sim	Sim	Não
Editar ou excluir canais de outros operadores	Sim	Não	Não
Visualizar painel de auditoria e logs do Redis	Sim	Não	Não
Acessar métricas financeiras de MRR e assinantes	Sim	Não	Não
Realizar upgrade para plano pago via Stripe	Sim	Sim	Sim
Gerenciar faturas e cancelar própria assinatura	Sim	Sim	Sim

Isolamento de dados por usuário: O sistema impede falhas de referência direta a objetos inseguros (IDOR). Todas as operações de leitura e modificação de perfis, exclusão de favoritos e gerenciamento de assinaturas injetam explicitamente a cláusula `WHERE usuario_id = %s` amarrada à identidade validada na sessão criptografada do usuário autenticado (`session['user_id']`). Um usuário jamais consegue visualizar faturas ou alterar dados de outro assinante.

Gestão de Segredos: Nenhuma chave secreta ou senha reside no código-fonte ou no repositório GitHub. O repositório armazena unicamente o arquivo de modelo `.env.example`. Durante a inicialização da stack no Portainer, as variáveis confidenciais reais (`SECRET_KEY`, `DB_PASSWORD`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `MINIO_ROOT_PASSWORD`) são injetadas diretamente na memória dos containers pelo daemon do Docker, ficando inacessíveis para o público externo.

3.7 Observabilidade, CI/CD e documentação da API

Em conformidade com a realidade do repositório implementado:

- **Documentação Interativa (Swagger/OpenAPI):** Foi implementado o Flasgger nos microsserviços do projeto (`catalog-service`, `auth-service` e `log-service`), fornecendo documentação interativa das rotas REST e seus parâmetros através do endpoint público `/apidocs/`.
- **Testes Automatizados:** O projeto conta com uma suíte abrangente de 23 testes unitários e de integração (`test_stripe_integration.py` e `test_openwave_integration.py`), cobrindo validação de payloads HMAC do Stripe, garantia de idempotência com deduplicação, travas de cotas na API (retorno 403), rebaixamento por inadimplência (`past_due`) e verificação de papéis RBAC.
- **Observabilidade dos Eventos:** O sistema não utiliza Prometheus ou Grafana; os eventos de negócio (bloqueios 403, criação de canais, processamento de webhooks) são registrados de forma contínua pelo `log-service` em Redis Streams e podem ser consultados pelo administrador no painel web `/admin/logs`. O projeto não possui pipeline de CI/CD automatizado via GitHub Actions.

3.8 Camadas IaaS, PaaS e SaaS do projeto

Camada	O que é no projeto OpenWave	Quem gerencia
IaaS	A máquina virtual Linux provisionada em nuvem para a disciplina, incluindo vCPU, memória RAM, disco e roteamento de rede IP.	Professor e Fatec Pompeia.
PaaS	A engine Docker e o Portainer para orquestração da stack; a rede interna bridge `app-network`; o banco de dados MariaDB externo (`35.226.64.52`); o cache Redis; o storage MinIO S3; e o proxy reverso com SSL/TLS sob o domínio `lapps.studio`.	Compartilhado: o aluno gerencia a configuração dos containers (`docker-compose.yml`), schemas e volumes; o professor gerencia a infraestrutura do MariaDB e o gateway proxy.
SaaS	O produto final **OpenWave** entregue aos ouvintes e feeders (streaming, limites e inteligência tática); e os provedores SaaS externos consumidos pelo sistema (Stripe para recorrência, Mailtrap para e-mails transacionais e GitHub para repositórios).	Aluno (regras de negócio e código do OpenWave); fornecedores externos (Stripe Inc., Mailtrap e GitHub gerenciam suas plataformas).

4 Quadro de entregas

Esta tabela consolida as atividades realizadas no 2º bimestre, extraídas automaticamente a partir das fichas de comprovação técnica apresentadas na seção a seguir.

Nº	Atividade	Data planejada	Data realizada	Situação
7	Plano Premium — cobrança de verdade com Stripe	09/10/2026	08/10/2026 21:13	entregue
8	SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve	16/10/2026	19/10/2026 19:43	entregue com atraso
9	Fork e plano de negócio	23/10/2026	23/10/2026 16:46	entregue
10	Seu negócio no ar — o domínio no lugar dos filmes	06/11/2026	06/11/2026 20:08	entregue
11	SaaS completo — planos, limites e assinatura com Stripe	13/11/2026	12/11/2026 19:46	entregue

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

Descrição	Implementação de plano pago em modo de teste: integração com Checkout Session da Stripe e webhook marcando usuário como premium no banco.
Data planejada	09/10/2026
Data realizada	08/10/2026 21:13
Situação	entregue
Evidência	GitHub — commit 5ba13ea + issue #6 + prints do checkout concluído com cartão 4242 e usuário premium com badge
Link	https://github.com/fezinhopatta/Tom-Hanks-api/commit/5ba13ea

O que foi feito. Foi integrada a API oficial do Stripe em modo de teste no catálogo de filmes. Desenvolveu-se a rota `/checkout` que instancia uma sessão de pagamento na Stripe via biblioteca Python e redireciona o cliente. Criou-se o endpoint público `/webhook/stripe` que processa o evento `checkout.session.completed`, atualizando a coluna `is_premium = 1` no banco de dados MariaDB. Na interface, os assinantes passaram a receber o selo visual de "Assinante Premium" e a liberação de favoritos ilimitados.

Dificuldades e como foram resolvidas. A principal dificuldade inicial consistiu no recebimento e validação dos webhooks do Stripe no ambiente local de desenvolvimento, resolvida mediante o uso do Stripe CLI para escuta de eventos e pelo correto roteamento do endpoint de notificação na stack do Portainer.

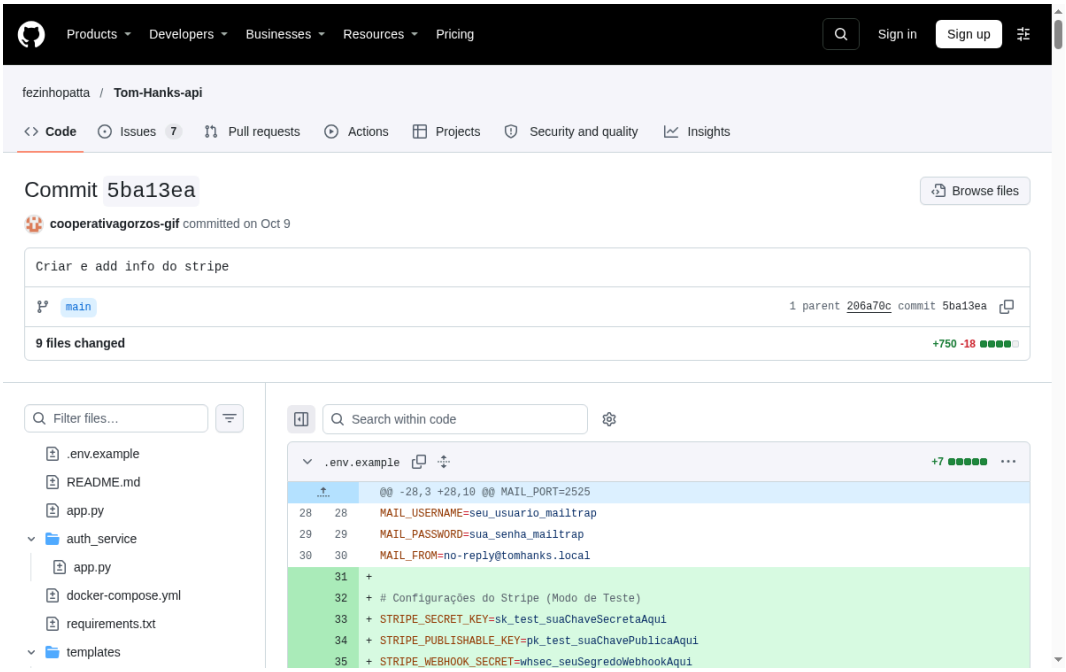


Figura 4: Commit 5ba13ea no repositório Tom-Hanks-api com hash e data visíveis

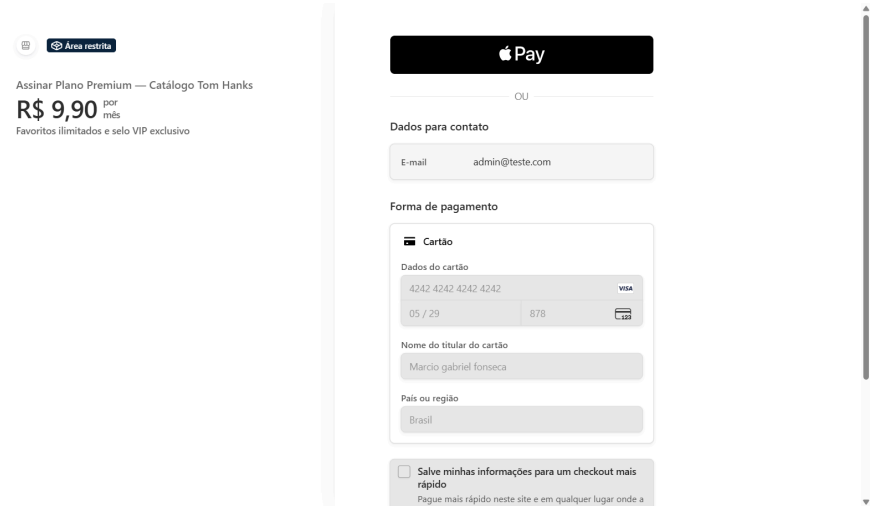


Figura 5: Sessão de Stripe Checkout concluída com o cartão de testes 4242

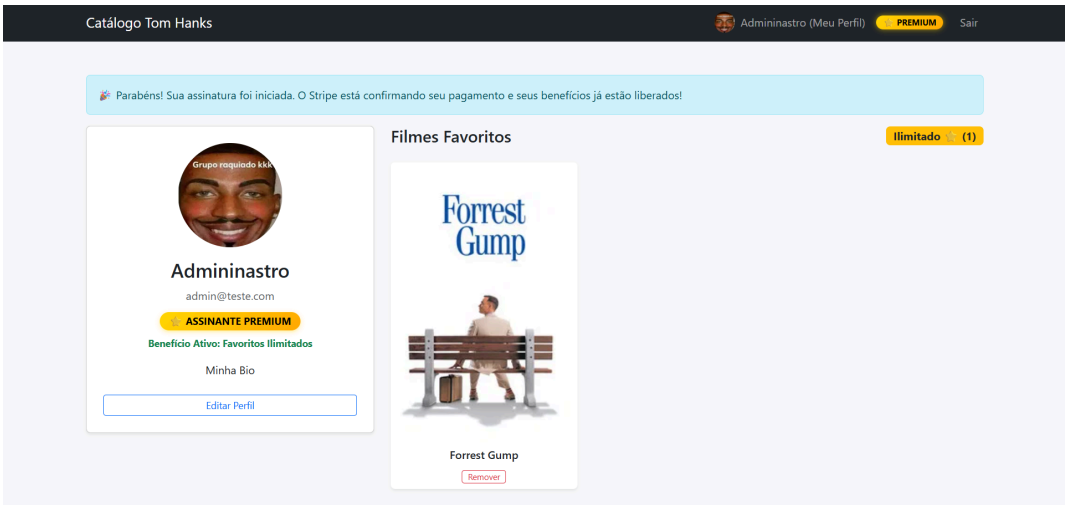


Figura 6: Catálogo exibindo usuário Admininastro promovido a Assinante Premium

5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve

Descrição	Mapeamento das camadas de nuvem do próprio projeto e elaboração da documentação reversa completa contendo especificação, arquitetura, regras de negócio e modelo de dados.
Data planejada	16/10/2026
Data realizada	19/10/2026 19:43
Situação	entregue com atraso
Evidência	GitHub — commit 89fdb5d + issue #7 + DOCUMENTACAO.md linkado no README
Link	https://github.com/fezinhopatta/Tom-Hanks-api/commit/89fdb5d

O que foi feito. Foi redigida a especificação técnica reversa de todo o sistema desenvolvido até então, consolidada no arquivo `DOCUMENTACAO.md` e referenciada de forma destacada no `README.md`. O documento contemplou a matriz de responsabilidades das camadas de nuvem (IaaS na VM da disciplina, PaaS com Docker/Portainer e SaaS na aplicação final), detalhamento da topologia dos microsserviços, diagramas de sequência, dicionário de dados do MariaDB e inventário das variáveis de ambiente.

Dificuldades e como foram resolvidas. A entrega foi concluída em 19/10/2026, com 3 dias de atraso em relação ao prazo estipulado (16/10), devido à profundidade necessária para catalogar e diagramar minuciosamente cada microsserviço pré-existente e mapear com precisão as fronteiras entre as camadas de computação em nuvem.

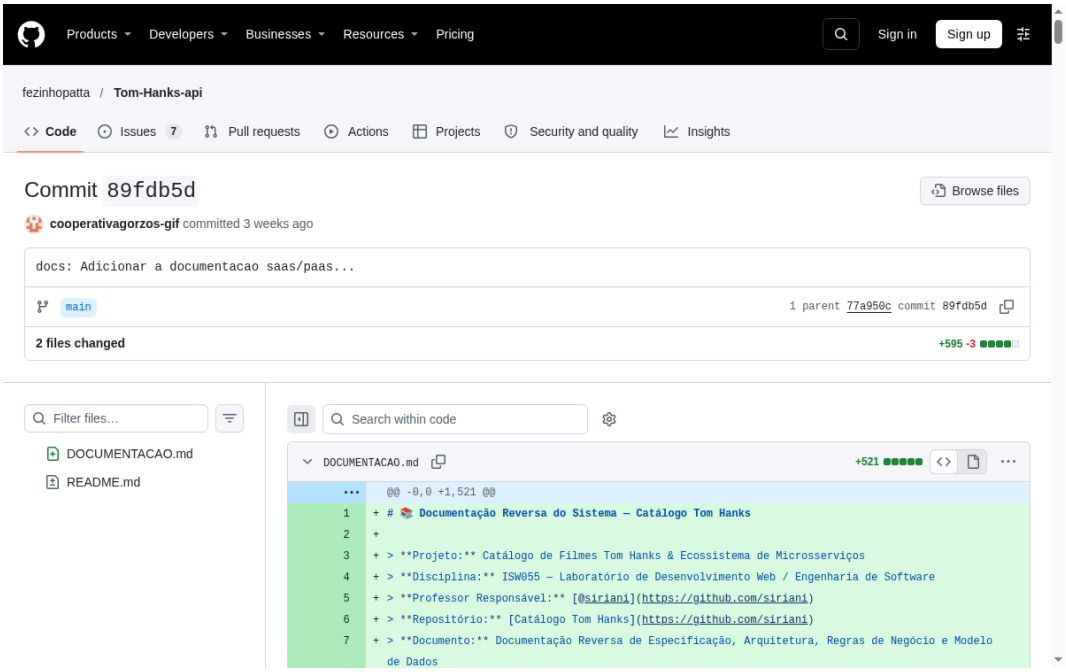


Figura 7: Commit 89fdb5d adicionando a documentação completa no repositório

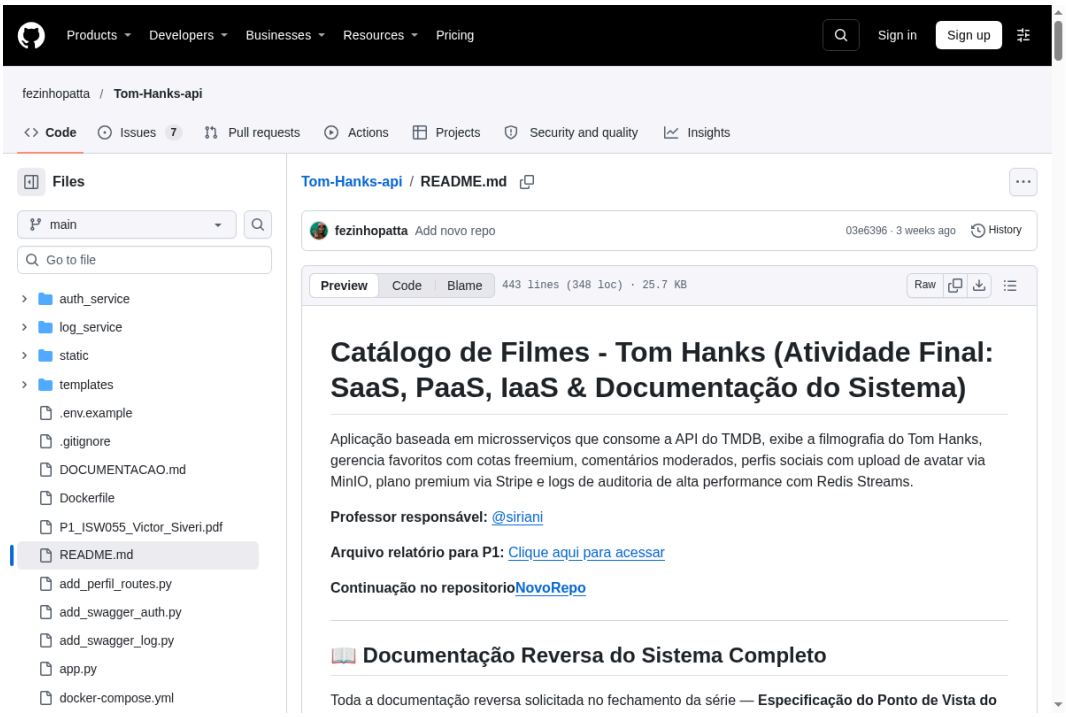


Figura 8: README.md exibindo o link e a estrutura da documentação reversa

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Criação do repositório novo preservando 100% do histórico anterior, elaboração do README com a proposta de valor do produto (OpenWave, dor, personas, Lean Canvas, modelo de receita) e abertura do backlog em issues.
Data planejada	23/10/2026
Data realizada	23/10/2026 16:46
Situação	entregue
Evidência	GitHub — issue #6 (\@siriani) + repositório TraumaTeam + README do produto + histórico espelhado + issues de backlog #1 a #5
Link	https://github.com/fezinhopatta/TraumaTeam/issues/6

O que foi feito. O repositório do produto **OpenWave** foi estabelecido a partir da duplicação integral do histórico do catálogo de filmes via clone bare e push mirror. O `README.md` foi completamente reformulado para apresentar o novo plano de negócio: dor do monitoramento de rádio de emergência, personas (rádio-operador voluntário **feeder** e cidadão/analista **listener**), Lean Canvas, modelo de monetização SaaS e plano de reaproveitamento da infraestrutura. Foram abertas e documentadas no GitHub as issues `#1` a `#5` correspondentes às sprints de modelagem, RBAC, MinIO, interface e Stripe.

Dificuldades e como foram resolvidas. O maior cuidado técnico foi garantir que o mirror do Git preservasse com fidelidade os hashes e timestamps de todos os commits realizados desde o primeiro bimestre, o que foi assegurado pela execução do comando `git push --mirror`.

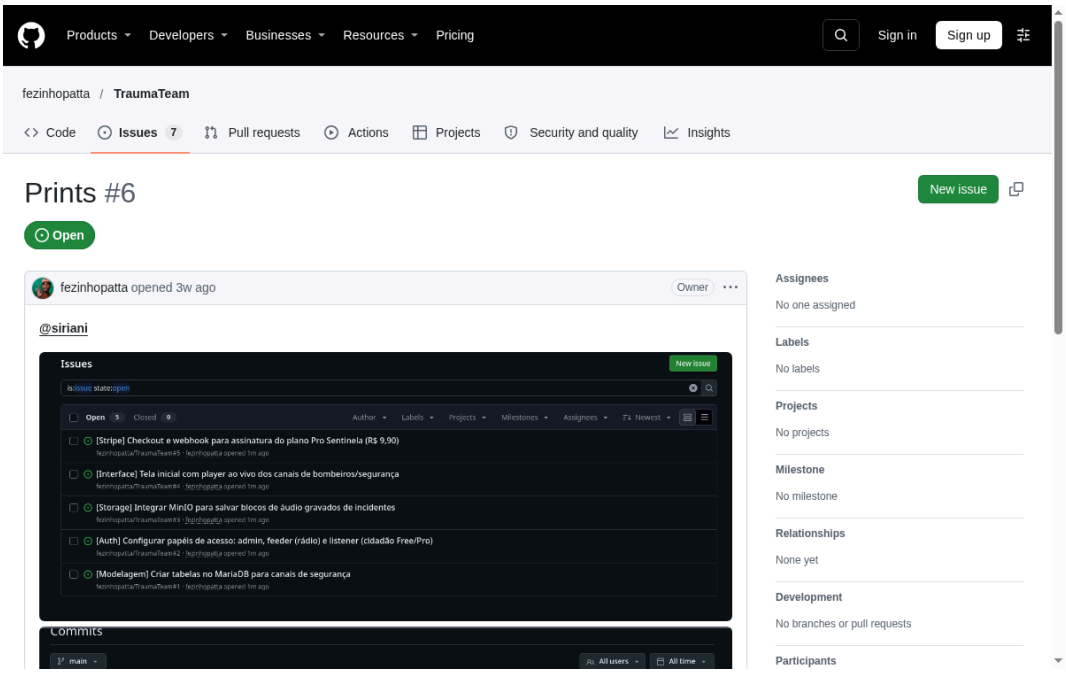


Figura 9: Issue 6 de entrega no repositório TraumaTeam mencionando o professor @siriani

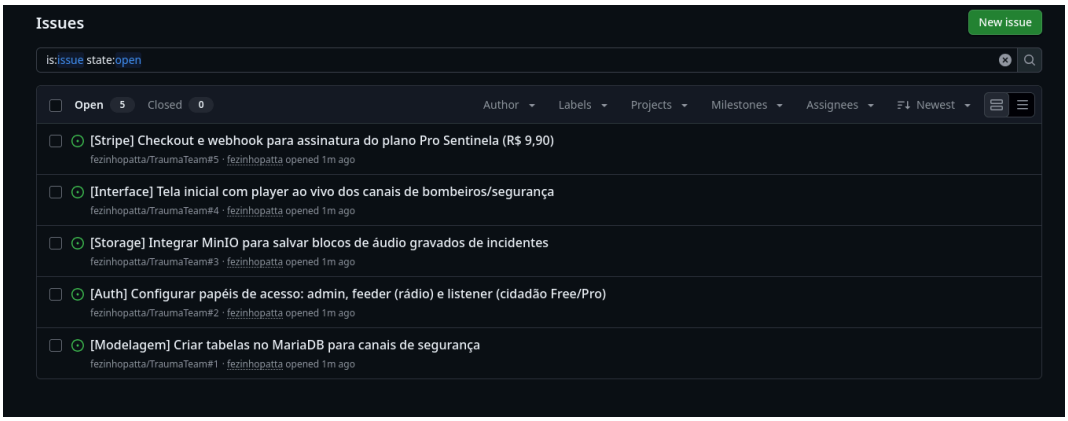


Figura 10: Backlog de issues (1 a 5) cadastradas no novo repositório

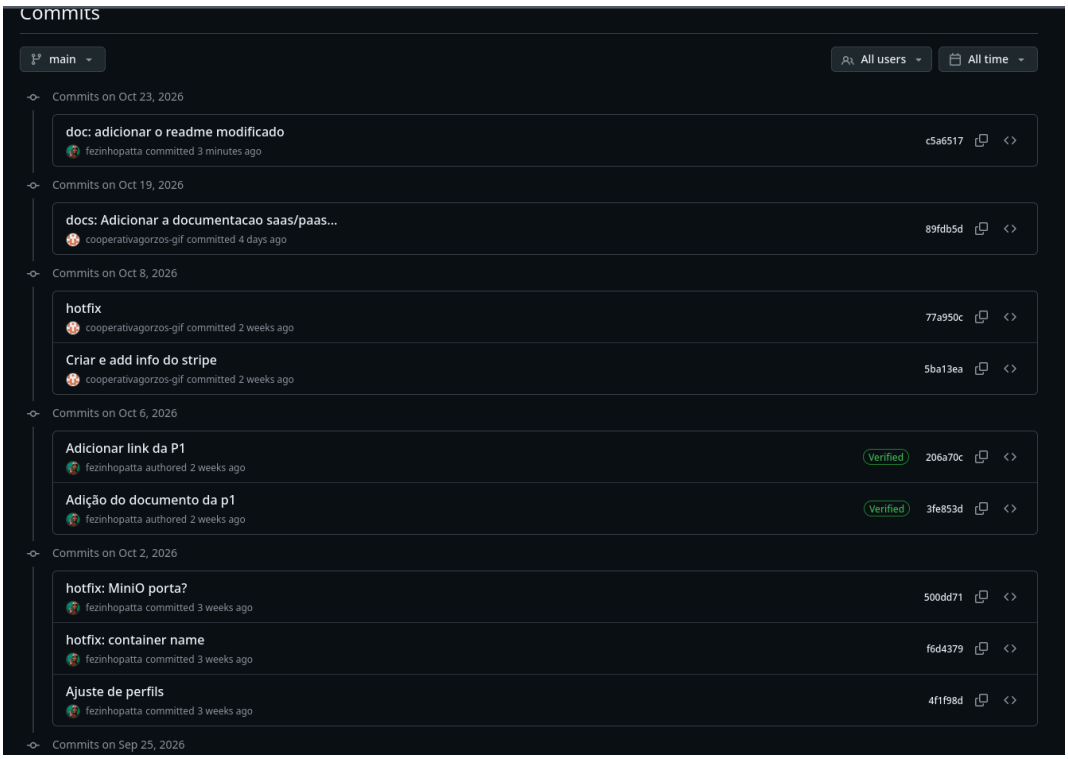


Figura 11: Histórico contínuo de commits preservado após o espelhamento do repositório

5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

Descrição	Migração completa da stack no Portainer para o repositório novo: substituição do catálogo de filmes pelo domínio do OpenWave, modelagem de canais, papéis RBAC (feeder/listener), MinIO, MariaDB, landing page e logs.
Data planejada	06/11/2026
Data realizada	06/11/2026 20:08
Situação	entregue
Evidência	GitHub — commit 02b6c35 + issue #7 (\@siriani) + URL no ar + prints da landing, dashboard, canais e logs de auditoria
Link	https://github.com/fezinhopatta/TraumaTeam/commit/02b6c35

O que foi feito. O domínio de filmes foi inteiramente substituído pelo negócio do OpenWave. A stack do Portainer foi reconfigurada para compilar o código do novo repositório. Foram implementadas as tabelas de canais e gravações no MariaDB, o controle de acesso RBAC diferenciando **feeder** (permissão de transmissão) e **listener** (permissão de escuta), upload de arquivos no bucket S3 do MinIO, streaming de áudio, landing page institucional e ingestão de eventos no `log-service` com persistência em Redis Streams. O commit de entrega foi finalizado e submetido em 06/11 às 20:08, e a issue de formalização com os prints de homologação foi registrada na manhã subsequente.

Dificuldades e como foram resolvidas. Ajustar as rotas de streaming e upload de áudio no MinIO para operarem de maneira transparente sob a rede interna do Docker exigiu padronizar as variáveis de ambiente `MINIO_ENDPOINT` e configurar a política de acesso no bucket `openwave-midias`.

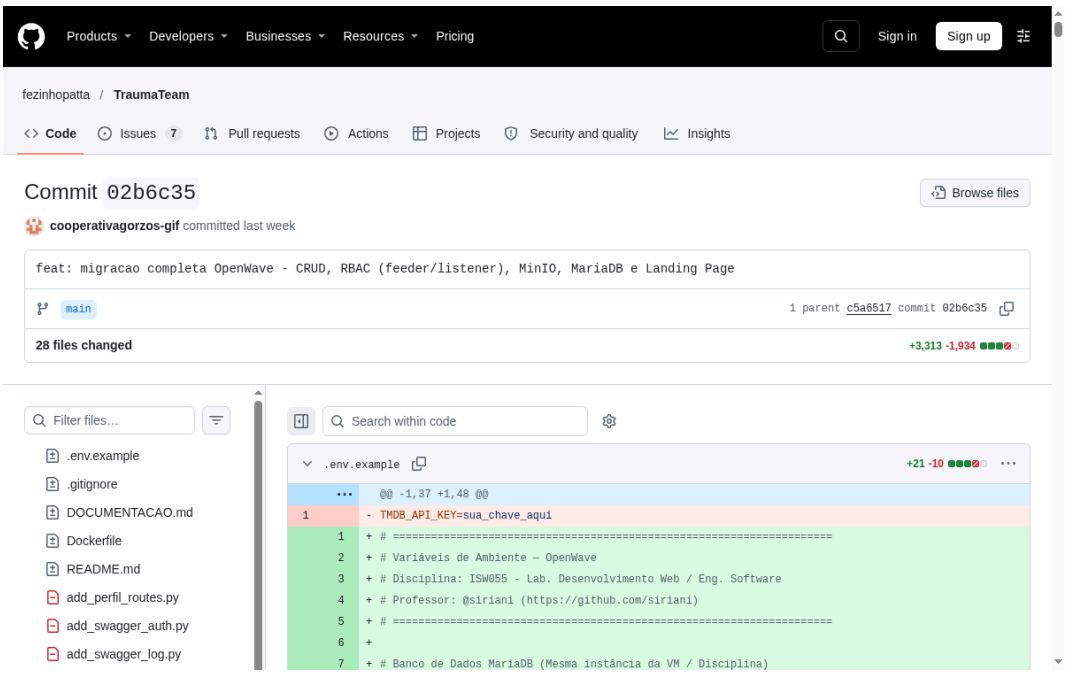


Figura 12: Commit 02b6c35 consolidando a migração definitiva para o OpenWave

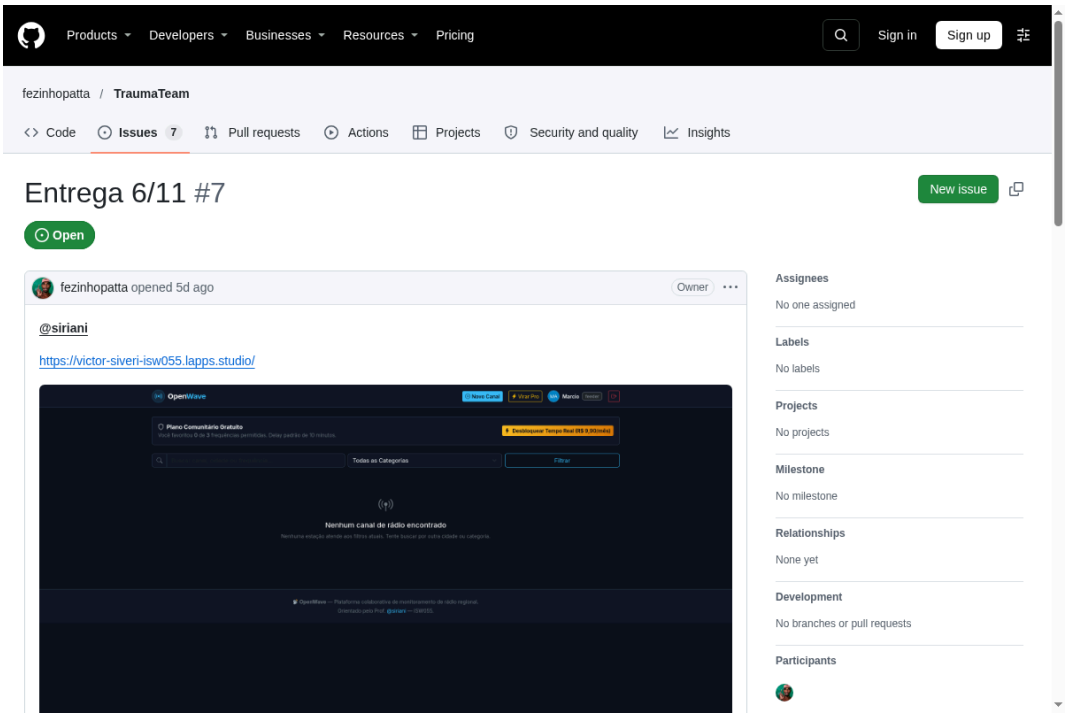


Figura 13: Issue 7 com prints do sistema em operação e menção a @siriani



Figura 14: Landing Page do OpenWave em produção no subdomínio lapps.studio

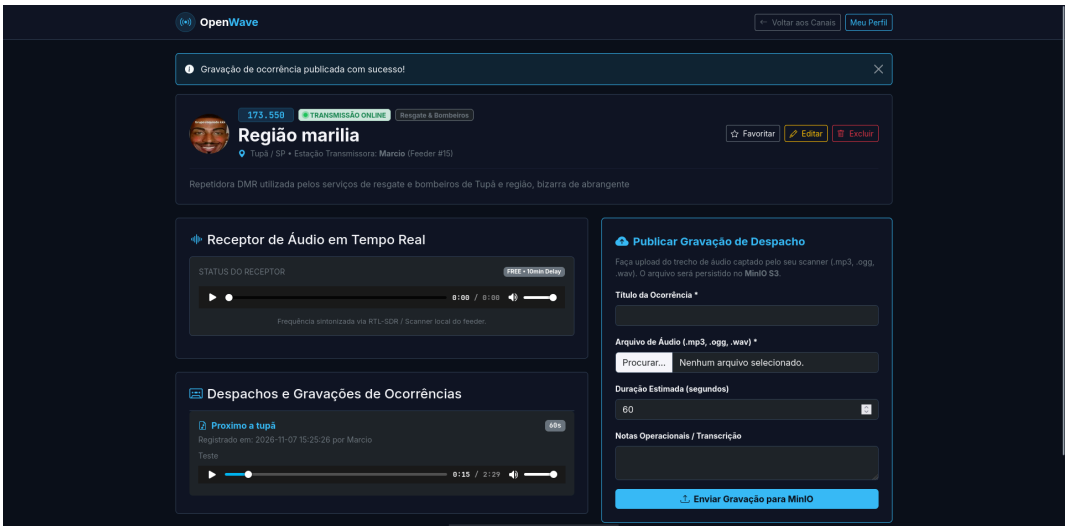


Figura 15: Dashboard operacional do Feeder e detalhe de canal com player de áudio do MinIO



Figura 16: Painel de auditoria do log-service alimentado em tempo real via Redis Streams

5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

Descrição	Evolução para o modelo SaaS com assinaturas recorrentes mensais no Stripe, validação de webhooks, proteção de idempotência, portal de autoatendimento, trava por requisição e painel financeiro.
Data planejada	13/11/2026
Data realizada	12/11/2026 19:46
Situação	entregue
Evidência	GitHub — commit ab70866 (branch ULTIMA) + prints do autoatendimento no perfil, tela educativa de cota atingida (403) e painel admin de MRR
Link	https://github.com/fezinhopatta/TraumaTeam/commit/ab70866

O que foi feito. A cobrança estática e pontual da atividade 7 foi transformada em uma arquitetura SaaS de recorrência mensal completa na branch `ULTIMA`. Foram criados os planos **Comunitário (Free)**,

Sentinela Pro (R$ 9,90/mês) e **Central Tática VIP** (R$ 29,90/mês). Implementou-se a verificação de direitos a cada requisição na API (`obter_dados_assinatura`), limitando usuários gratuitos a no máximo 3 frequências favoritas e gerando a página de bloqueio educativo HTTP 403 (`limite.html`) quando a cota é atingida. Foram criadas rotas de autoatendimento para o assinante: integração com o Stripe Customer Portal (`/assinatura/portal`), troca autônoma de plano e cancelamento direto (`/assinatura/cancelar`). O webhook `/webhook/stripe` passou a tratar os 4 eventos essenciais do ciclo de vida com validação criptográfica HMAC SHA-256 e proteção contra duplicações baseada na tabela `stripe_eventos`. Criou-se também o painel administrativo de assinaturas (`/admin/assinaturas`) exibindo MRR em tempo real, contagem de assinantes e histórico auditável de eventos recebidos.

Dificuldades e como foram resolvidas. Garantir que retransmissões automáticas da rede do Stripe não provocassem execuções repetidas exigiu desenhar um mecanismo rigoroso de idempotência, solucionado com a tabela relacional `stripe_eventos`. A trava de inadimplência foi implementada com rebaixamento imediato dos benefícios na ocorrência do evento `invoice.payment_failed` ou `past_due`.

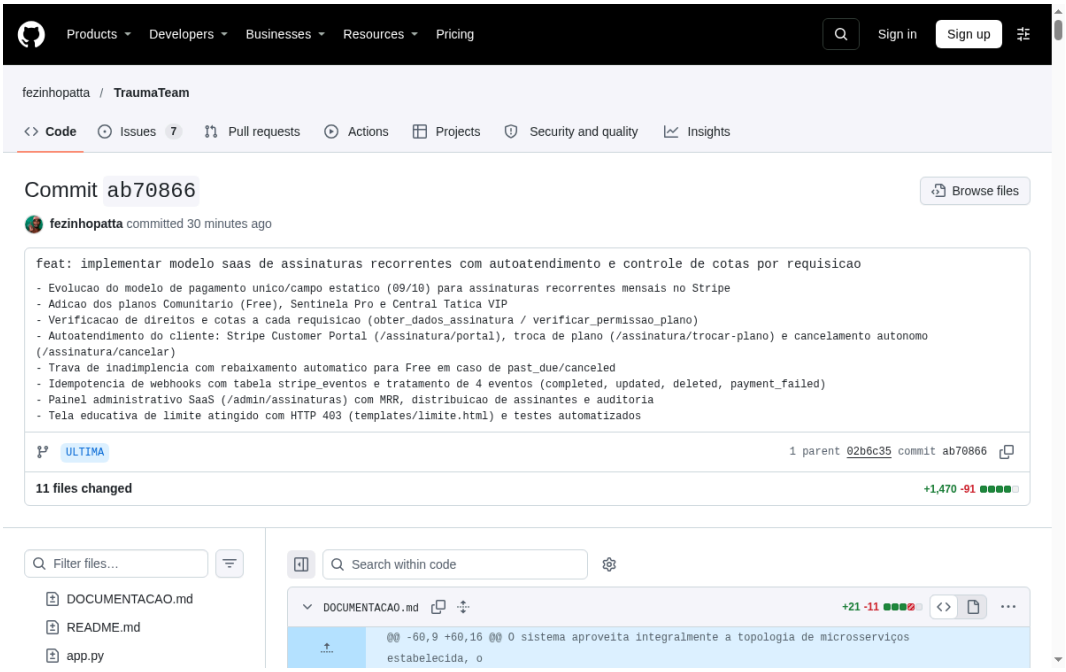


Figura 17: Commit ab70866 na branch ULTIMA implementando o modelo SaaS completo

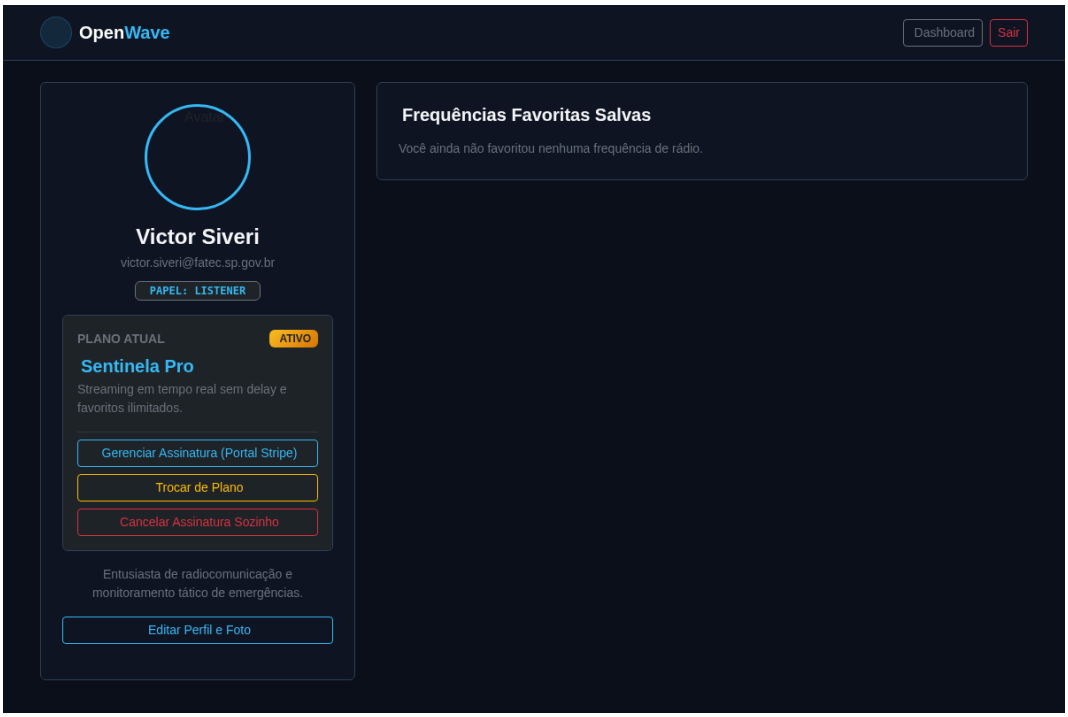


Figura 18: Perfil do assinante com controles autônomos (Portal Stripe, Troca de Plano e Cancelamento)



Figura 19: Bloqueio educativo HTTP 403 com aviso de cota máxima atingida no Plano Gratuito

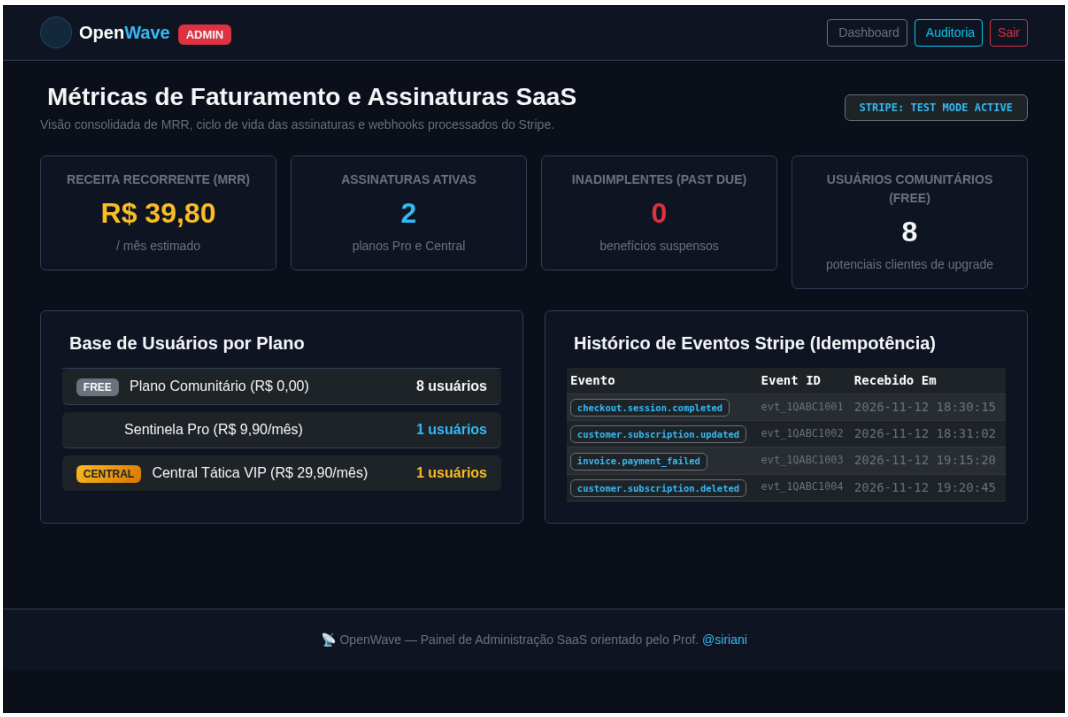


Figura 20: Painel Administrativo SaaS consolidando MRR, distribuição de planos e histórico de webhooks

6 Considerações finais

O desenvolvimento do OpenWave ao longo do segundo bimestre proporcionou um aprendizado prático e profundo sobre a engenharia de software aplicada a ambientes de computação em nuvem. A evolução a partir de um catálogo monolítico e pontual até a concepção de um ecossistema desacoplado de microsserviços evidenciou como decisões de arquitetura — como segregação de redes no Docker, RBAC granular e persistência externa — impactam diretamente a escalabilidade, resiliência e segurança de um produto digital.

O maior desafio técnico concentrou-se na orquestração assíncrona do ciclo de vida das assinaturas com o gateway Stripe. Lidar com eventos descentralizados que trafegam pela internet pública demandou implementar validação rigorosa de integridade criptográfica (HMAC SHA-256), proteção de idempotência contra concorrência e um modelo estrito de governança de cotas que valida os direitos do cliente a cada requisição sem onerar a latência do banco de dados relacional.

Para que o OpenWave evolua de um projeto acadêmico de excelência para uma plataforma comercial em escala nacional, os passos futuros compreendem a implementação de uma esteira automatizada de CI/CD via GitHub Actions com deploy contínuo, a instrumentação de telemetria e métricas de desempenho com Prometheus e Grafana, e a integração direta de estações de recepção de rádio baseadas em software definido (SDR) via WebRTC.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados.

Victor Hugo Cano Siveri

Pompeia, 13/11/2026